Practical Manual

Docker Fundamentals, Containerization, AWS ECR and ECS Deployment

Lab Objective

By completing this practical exercise, you will learn how to:

- Understand Docker fundamentals
- Run and manage Docker containers
- Create custom Docker images
- Build a web application container
- Push Docker images to AWS Elastic Container Registry (ECR)
- Deploy containerized applications on AWS ECS Fargate

Section 1: Verify Docker Environment

Step 1: Open AWS CloudShell

1. Login to AWS Console.
2. Open CloudShell from the AWS Console toolbar.
3. Select your preferred AWS Region.

Step 2: Verify Docker Installation

Execute:

```
docker --version
```

Expected output:

```
Docker version xx.xx.xx
```

Check Docker service information:

```
docker info
```

Observe:

- Docker Engine details
- Storage driver
- Running containers count
- Available images

This confirms Docker is available and operational.

Section 2: Understanding Docker Containers

Step 3: Run Your First Container

Execute:

```
docker run hello-world
```

What happens?

1. Docker checks local images.
2. Downloads hello-world image if absent.
3. Creates a container.
4. Executes it.
5. Displays a success message.

This verifies Docker can pull and run containers successfully.

Step 4: Launch an Interactive Ubuntu Container

Execute:

```
docker run -it ubuntu /bin/bash
```

You are now inside a Linux container.

Try:

```
ls
uname -a
pwd
```

Observe:

- The container has its own filesystem.
- The container has its own process environment.

Exit the container:

```
exit
```

Step 5: View Containers

Display running containers:

```
docker ps
```

Display all containers:

```
docker ps -a
```

Observe:

- Container IDs
- Image names
- Status
- Creation times

Section 3: Working with Docker Images

Step 6: Search Docker Hub Images

Execute:

```
docker search alpine
```

Observe available Alpine Linux images.

Step 7: Download an Image

Pull Alpine Linux:

```
docker pull alpine
```

Step 8: Run a Command Inside an Image

Execute:

```
docker run alpine echo "Docker is awesome!"
```

Docker:

1. Creates a temporary container.
2. Executes the command.
3. Removes the container after completion.

Step 9: View Downloaded Images

```
docker images
```

Observe:

- Repository
- Tag
- Image ID
- Size

Section 4: Creating Your First Dockerized Web Application

Step 10: Create Project Directory

```
mkdir ~/docker-lab
cd ~/docker-lab
```

Step 11: Create a Simple Web Page

```
echo '<h1>Hello from Docker!</h1>' > index.html
```

Verify:

```
cat index.html
```

Step 12: Create a Dockerfile

Create a Dockerfile:

```
echo -e 'FROM nginx:alpine\nCOPY . /usr/share/nginx/html' > Dockerfile
```

Verify:

```
cat Dockerfile
```

Expected:

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

Understanding the Dockerfile:

Instruction	Purpose
FROM	Uses nginx Alpine image as base
COPY	Copies website files into nginx document root

Section 5: Build a Docker Image

Step 13: Build the Image

Execute:

```
docker build -t my-web-app .
```

Explanation:

- docker build → Builds image
- -t → Assigns image name
- my-web-app → Image name
- . → Current directory context

Step 14: Verify Image Creation

```
docker images
```

You should see:

```
my-web-app
```

listed among available images.

Section 6: Run the Custom Image

Step 15: Start a Container

```
docker run -d -p 8080:80 my-web-app
```

Explanation:

Option	Purpose
-d	Run in background
-p	Map ports
8080	Host port
80	Container port

Step 16: Verify Running Containers

```
docker ps
```

You should see:

```
my-web-app
```

running.

Step 17: Preview the Application

In CloudShell:

```
Actions → Preview → Preview Port 8080
```

Expected output:

```
Hello from Docker!
```

This confirms your web application is being served from inside the container.

Section 7: Create an AWS ECR Repository

What is ECR?

AWS Elastic Container Registry (ECR) is a private Docker image repository service.

It stores Docker images securely within AWS.

Step 18: Create Repository

Set repository name:

```
ecr_repo_name=sample-web-app
```

Create repository:

```
aws ecr create-repository \
--repository-name $ecr_repo_name \
--image-scanning-configuration scanOnPush=true \
--region us-east-1
```

Observe the output.

Important:

Copy and save the Repository URI.

Example:

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/sample-web-app
```

Section 8: Authenticate Docker with ECR

Step 19: Get AWS Account ID

Execute:

```
aws sts get-caller-identity --query Account --output text
```

Save the account number.

Step 20: Login to ECR

Execute:

```
aws ecr get-login-password \
--region us-east-1 | \
docker login \
--username AWS \
--password-stdin \
<account-id>.dkr.ecr.us-east-1.amazonaws.com
```

Expected:

```
Login Succeeded
```

Docker can now push images to ECR.

Section 9: Tag Docker Image for ECR

Step 21: Build Production Image

Create a new application directory:

```
mkdir ~/sample-docker-app
cd ~/sample-docker-app
```

Create webpage:

```
echo '<!DOCTYPE html><html><body><h1>Hello from Docker!</h1></body></html>' >
index.html
```

Create Dockerfile:

```
echo -e 'FROM nginx:alpine\nCOPY . /usr/share/nginx/html' > Dockerfile
```

Build image:

```
docker build -t sample-web-app .
```

Step 22: Tag Image

Set repository URI:

```
repo_uri=<your-repository-uri>
```

Tag image:

```
docker tag sample-web-app:latest $repo_uri:latest
```

Verify:

```
docker images
```

You should see the ECR-tagged image.

Section 10: Push Image to AWS ECR

Step 23: Push Image

```
docker push $repo_uri:latest
```

Observe:

- Layer upload progress
- Image digest
- Push completion

Step 24: Verify Image in ECR

Using CLI:

```
aws ecr list-images \
--repository-name sample-web-app
```

Or:

1. Open AWS Console
2. Navigate to ECR
3. Open repository
4. Verify image tag "latest"

The image is now stored in AWS ECR.

Section 11: Deploy Image to ECS Fargate

Step 25: Verify Image Availability

Open:

```
AWS Console → ECR
```

Confirm:

```
sample-web-app:latest
```

exists.

Step 26: Create ECS Cluster

Navigate:

```
AWS Console → ECS
```

Create Cluster:

```
Cluster Name: demo-cluster
Launch Type: Fargate
```

Create cluster.

Step 27: Create Task Definition

Configure:

```
Task Definition Name: sample-web-task
Memory: 0.5 GB
vCPU: 0.25
```

Container Configuration:

```
Container Name: web
Image URI: <ECR URI>:latest
Port: 80
```

Create task definition.

Step 28: Create ECS Service

Configuration:

```
Launch Type: Fargate
Service Name: web-service
Number of Tasks: 1
```

Networking:

```
Default VPC
2 Public Subnets
Public IP Enabled
Security Group allowing HTTP (Port 80)
```

Create service.

Section 12: Access the Application

Step 29: Locate Public IP

Navigate:

```
ECS Cluster
→ Tasks
→ Select Task
→ ENI
→ Public IPv4 Address
```

Copy the IP.

Step 30: Open Browser

Visit:

```
http://<public-ip>
```

Expected output:

```
Hello from Docker!
```

Your containerized application is now running on AWS ECS Fargate.

Section 13: Cleanup

Delete ECS resources:

- ECS Service
- ECS Cluster
- Task Definitions (optional)

Delete ECR Repository:

```
aws ecr delete-repository \
--repository-name sample-web-app \
--force
```

Delete local Docker images:

```
docker rmi my-web-app
```

```
docker rmi $repo_uri:latest
```

Clean unused Docker resources:

```
docker system prune -f
```

Key Concepts Learned

1. Docker Images
2. Docker Containers
3. Dockerfile Creation
4. Image Build Process
5. Port Mapping
6. Docker Hub Images
7. AWS ECR Repositories
8. Docker Authentication to ECR
9. Image Tagging
10. Image Push Operations
11. ECS Fargate Deployment
12. Containerized Web Application Hosting